UNIVERSIDAD DE BURGOS

Grado en Ingeniería Informática

Memoria de Práctica I: Refactorización de una Planta Industrial con POO

Curso: 2026-2027 **Fecha de Entrega:** 4 de noviembre de 2026

1. INTRODUCCIÓN Y PLANTEAMIENTO

El objetivo de esta práctica ha sido refactorizar el código inicial de una simulación de planta industrial desarrollada sobre el motor físico **MuJoCo**, utilizando **OpenCV** para el procesamiento de visión y **NumPy** para los cálculos matriciales. El código de partida funcionaba correctamente pero estaba programado de forma totalmente procedural: la lógica de transporte, las cámaras, el horno y los controles estaban mezclados en un bucle principal (`run()`) y dependían de banderas y variables globales.

Para solucionar esto y preparar una base de código limpia para la Práctica II, se ha rediseñado la aplicación dividiendo las responsabilidades en clases independientes ubicadas dentro del directorio `src/`, manteniendo `main.py` en la raíz como único punto de entrada.

2. ESTRUCTURA DE CLASES E IMPLEMENTACIÓN

Se han creado las clases mínimas solicitadas en el enunciado, asegurando que cada una administre su propio estado interno:

2.1. Clase CintaTransportadora (src/cinta_transportadora.py)

Sustituye el uso de variables globales (como `BANDERA_CINTA_1`) por atributos de instancia (`self.activa`, `self.velocidad`, `self.numero`). Cada cinta conoce su dirección de movimiento en el plano XY según su identificador (del 1 al 5):

- **Cintas 1 a 4:** Forman el circuito cerrado principal (inferior, derecha, superior e izquierda).
- **Cinta 5:** Es la rama de desvío hacia la boca del horno industrial.

El método `calcular_fuerza(pos, vel)` calcula los vectores de fuerza requeridos por MuJoCo para mover la pieza a lo largo del carril o aplicar fuerza de frenado cuando la cinta se detiene.

2.2. Clase CamaraInspeccion (src/camara_inspeccion.py)

Se encarga de la adquisición de frames desde MuJoCo (`capturar()`) y de su procesamiento con OpenCV. Para la detección de formas en `detectar_forma()` se realiza una conversión a escala de grises, filtrado Gaussiano, umbralizado binario y extracción de contornos mediante `cv2.findContours()`.

La geometría de la pieza se clasifica combinando la aproximación poligonal de Douglas-Peucker (`cv2.approxPolyDP`) y la circularidad del contorno:

- **Cilindros / Círculos:** Contornos con más de 6 vértices y factor de circularidad superior a 0.78.
- **Cajas / Cubos:** Contornos aproximados a 4 vértices con ángulos rectos.

2.3. Control Reactivo: Variable objeto_detectado y Parada Automática

Tal y como exige el apartado 4.4 del enunciado, la clase `CamaraInspeccion` mantiene la variable booleana `self.objeto_detectado`. Cuando el algoritmo de visión encuentra una pieza dentro del área de inspección, esta variable pasa a `True` y se invoca la parada de la cinta asociada:

```
def actualizar(self, renderer, data):
    imagen = self.capturar(renderer, data)
    self.forma_detectada = self.detectar_forma(imagen)
    self.objeto_detectado = (self.forma_detectada is not None)

    # Parada automática cuando se detecta una pieza en la zona de inspección
    if self.objeto_detectado and self.cinta.esta_activa():
        self.cinta.detener()

    return self._superponer_info(imagen)
```

2.4. Clase Horno (src/horno.py)

Gestiona el conteo de piezas procesadas (`numero_piezas`) y la temperatura del sistema (`temperatura`). La temperatura aumenta dinámicamente cuando hay piezas en su interior y disminuye cuando está vacío.

Para reflejar el estado térmico en la simulación 3D de MuJoCo, el método `actualizar_color(model)` modifica el parámetro `geom_rgba` de la geometría interna del horno (`oven_interior`):

- **T < 50 °C:** Color azul `[0.1, 0.3, 1.0, 1.0]` (Horno frío / reposo).
- **50 °C ≤ T < 100 °C:** Color naranja `[1.0, 0.6, 0.1, 1.0]` (Temperatura nominal).
- **T ≥ 100 °C:** Color rojo `[1.0, 0.1, 0.1, 1.0]` (Temperatura elevada).

2.5. Clase ObjetoIndustrial (src/objeto_industrial.py)

Mantiene el seguimiento de cada una de las piezas que circulan por la planta. Almacena sus direcciones de memoria en MuJoCo (`qpos_addr`, `dof_addr`), su geometría y detecta en qué segmento del circuito se encuentra en cada paso de tiempo.

2.6. Clase Coordinadora PlantaIndustrial (src/planta.py)

Agrupa todos los componentes y sustituye la función procedural `run()`. Administra el bucle principal de simulación (a 500 Hz), invoca la actualización de cámaras y ventanas de OpenCV, gestiona los eventos de teclado (teclas 1..5 para cintas, P para pausa, R para reset, +/- para temperatura) y coordina el estado del robot Franka Panda.

3. ESTRUCTURA DE ARCHIVOS DEL PROYECTO

El proyecto se ha organizado respetando la separación entre el punto de entrada y los módulos internos:

```
planta_industrial/
├── main.py                    # Punto de entrada principal
├── requirements.txt           # Dependencias del proyecto
└── src/
    ├── __init__.py
    ├── config.py              # Parámetros numéricos y configuración
    ├── cinta_transportadora.py # Clase CintaTransportadora (direcciones 1 a 5)
    ├── camara_inspeccion.py   # Clase CamaraInspeccion y OpenCV
    ├── objeto_industrial.py   # Clase ObjetoIndustrial
    ├── horno.py               # Clase Horno y control térmico
    ├── planta.py              # Clase PlantaIndustrial (Coordinador)
    └── robot.py               # Cinemática inversa del robot Franka Panda
```

4. PRUEBAS REALIZADAS Y VERIFICACIÓN

Punto de Prueba	Resultado Observado	Estado
Arranque de MuJoCo y visor	Carga correcta del modelo 3D y entorno pasivo de MuJoCo.	Correcto
Control de cintas individuales	Conmutación de marcha/paro con las teclas 1 a 5.	Correcto
Mosaico de cámaras OpenCV	Apertura de la ventana con las 5 cámaras e información de estado.	Correcto
Detección de formas y parada	`objeto_detectado` pasa a `True` y la cinta se detiene al ver la pieza.	Correcto
Variación térmica del horno	El color del horno cambia de azul a naranja/rojo en función de la temperatura.	Correcto
Cinemática inversa del robot	El robot ejecuta las tareas de recogida y transferencia a la Cinta 5 sin problemas.	Correcto

5. CONCLUSIONES

La refactorización realizada ha permitido eliminar por completo la dependencia de variables globales, dejando una arquitectura modular y limpia. Cada componente tiene clara su responsabilidad, lo que facilitará la implementación de las tareas avanzadas de control y visión requeridas para la Práctica II.

Enlace al repositorio GitHub con el código completo del proyecto:
https://github.com/alejandrovenanciocalvino-cmd/Practica_1_Alejandro_Venancio_POO